

PP-LCNet_x1_0_doc_ori

Runtime Benchmark Report

Nepali Financial Document Orientation Classification

Date	2026-05-05
Author	prabin.lamichhane@amniltech.com
Ticket	#2963 — Amnil Research Board
Repo	github.com/prabinlamichhane-cell/research-page-orientation
Model	PP-LCNet_x1_0_doc_ori (PP-StructureV3, Paddle 3.x PIR)

1. Objective

Benchmark PP-LCNet_x1_0_doc_ori — PaddlePaddle's lightweight document orientation classifier — across three inference runtimes on a dataset of real Nepali financial documents. The goal is to identify the optimal runtime for production deployment.

Runtimes evaluated:

- **PaddlePaddle** (native Paddle 3.x PIR format) — baseline
- **ONNX Runtime** — converted via paddle2onnx (opset 17)
- **HuggingFace Optimum ORT** — same ONNX model via ORTModelForImageClassification

2. Dataset

Property	Value
Source PDFs	9 real Nepali financial documents
Document types	Audit reports, microfinance annual reports, management docs
Total images	552 (138 per class)
Classes	4 — 0°, 90°, 180°, 270°
Class balance	Perfectly balanced (25% each)
Messy/degraded images	210 (38%) — synthetically augmented
Messy transforms	Skew, shadow, ink bleed, fold, noise, blur, JPEG

Data quality note: All source PDFs were manually verified to ensure every source page is at true 0° before synthetic rotation. Landscape financial tables stored at 90° in PDF metadata were corrected to portrait. An earlier run without this correction yielded 89.1% accuracy; after correction accuracy rose to 96.9%, confirming label integrity as the most critical data pipeline step.



Figure 1 — Sample pages from collected Nepali financial PDFs

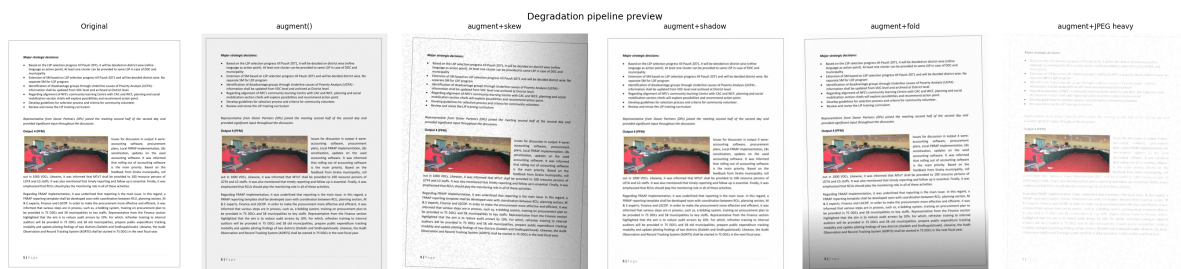


Figure 2 — Degradation pipeline: original → augmented → messify variants

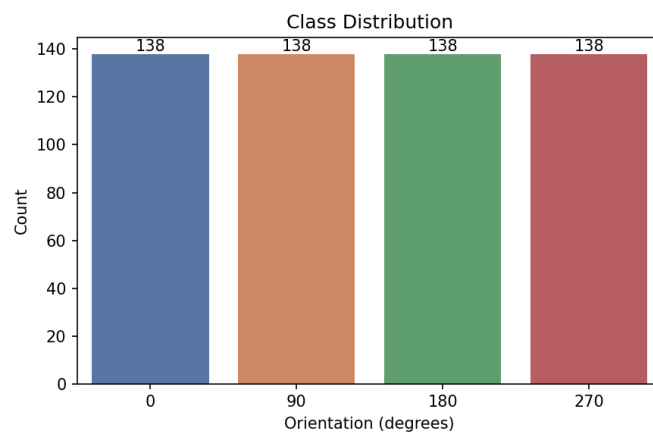


Figure 3 — Class distribution (balanced)

3. Results Summary

Runtime	Accuracy	Avg Latency	P50	P95	Throughput
PaddlePaddle	96.92%	39.53 ms	39.42 ms	46.54 ms	25.3 img/s
ONNX Runtime	96.92%	34.14 ms	33.53 ms	43.87 ms	29.3 img/s
Optimum ORT	96.92%	32.00 ms	32.23 ms	39.15 ms	31.2 img/s

Prediction agreement: 100% across all 3 runtimes — conversion is numerically lossless.

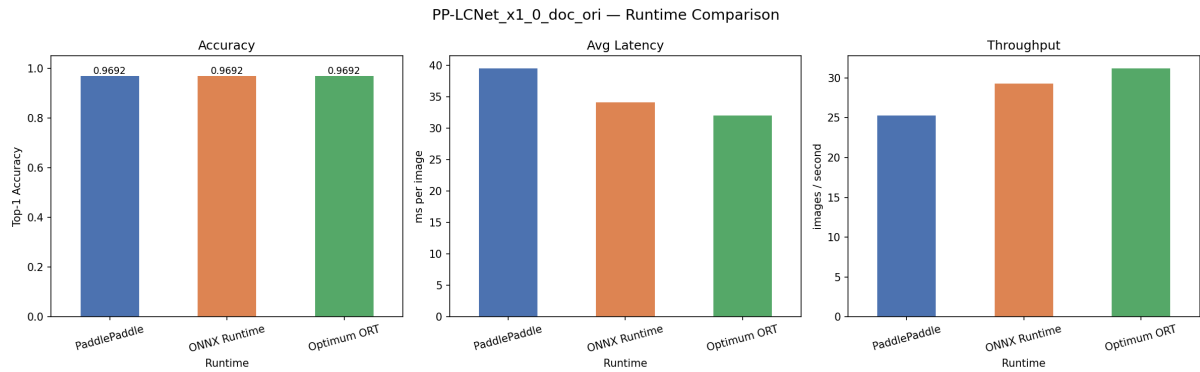


Figure 4 — Accuracy, avg latency and throughput comparison

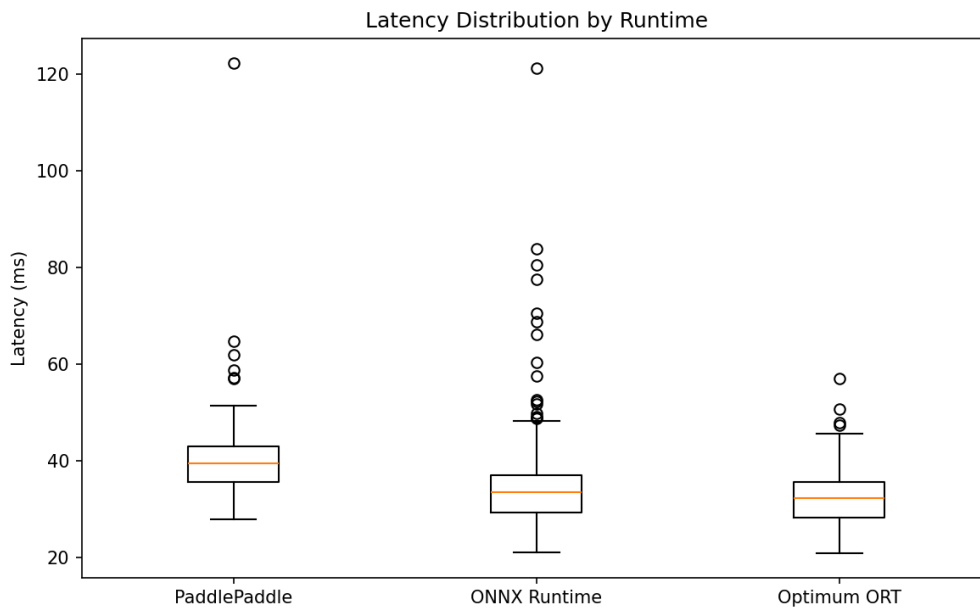


Figure 5 — Latency distribution by runtime

4. Per-class Performance

Identical across all runtimes (100% prediction agreement).

Class	Precision	Recall	F1-Score	Support
0°	0.9577	0.9855	0.9714	138
90°	0.9712	0.9783	0.9747	138
180°	0.9704	0.9493	0.9597	138
270°	0.9779	0.9638	0.9708	138
Overall	0.9693	0.9692	0.9692	552

Clean vs Messy Accuracy

Condition	Accuracy
Clean images	0.9883 (98.8%)
Messy / degraded images	0.9381 (93.8%)

Top Confusion Pairs

True	Predicted	Count
180°	0°	6
90°	270°	3
270°	90°	3
0°	180°	2
270°	180°	2
180°	90°	1

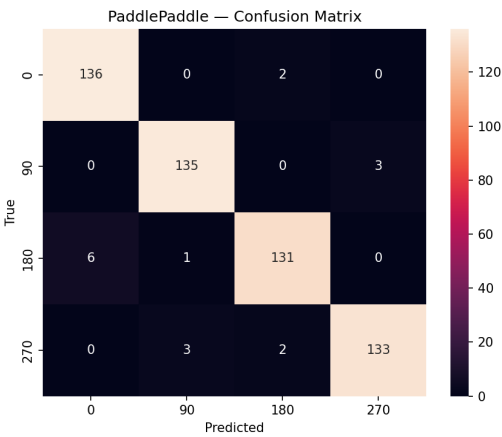


Figure 6 — Confusion matrix (identical for all runtimes)

5. Key Findings

1. **All runtimes produce identical predictions** — paddle2onnx at opset 17 is numerically lossless.
2. **ONNX Runtime is 14% faster** than native PaddlePaddle (34.14ms vs 39.53ms).
3. **Optimum ORT is 19% faster** than Paddle and 6% faster than raw ORT — HF wrapper adds no measurable overhead.
4. **96.9% out-of-the-box accuracy** on Nepali financial documents with no domain fine-tuning.
5. **5% accuracy drop on messy images** (93.8% vs 98.8%) — the primary gap to close with fine-tuning.
6. **Label integrity is critical** — mislabeled source pages caused 7.8% artificial accuracy drop in the first run.

6. Recommendation

Use ONNX Runtime for production deployment.

Criterion	Recommendation
Accuracy	Any runtime (identical)
Speed	Optimum ORT or ONNX Runtime
Dependency weight	ONNX Runtime (lighter than Optimum)
HuggingFace pipeline integration	Optimum ORT
Standalone deployment	ONNX Runtime

- **Standalone:** onnxruntime + model.onnx — minimal dependencies, 19% faster than Paddle.
- **HuggingFace pipeline:** Optimum ORT — fastest option, clean API.
- **Drop PaddlePaddle** from production — heaviest dependency, slowest, no accuracy benefit.

7. Fine-tuning — Is It Worth It?

96.9% is a strong out-of-the-box baseline. The remaining errors cluster around $180^\circ \leftrightarrow 0^\circ$ confusions on symmetric layouts and degraded scan robustness. Fine-tuning is feasible and recommended if production accuracy requirements exceed 98% or deployment targets heavily degraded documents.

Expected gains from fine-tuning:

Approach	Expected Accuracy	Effort
No fine-tuning (current)	~97%	Done
Feature extraction (freeze backbone, train head)	~98-99%	Low — 1-2 days
Full fine-tuning	~99%+	Medium — 3-5 days
Full fine-tuning + larger dataset (500+ pages)	~99%+ on messy	High

Bottleneck is data, not model capacity. PP-LCNet_x1_0 has sufficient capacity for 4-class orientation. Currently 138 source pages from 9 PDFs. Collecting 500+ real Nepali pages — including genuinely degraded scans from cooperatives, NGOs, and municipalities — will drive the largest accuracy improvement.

Suggested fine-tuning approach:

1. Collect 500+ real Nepali financial pages including degraded scans.
2. Rotate $\times 4 \rightarrow$ 2000+ labeled images.
3. Feature extraction: freeze PP-LCNet backbone, retrain 4-class head only.

4. If accuracy plateaus below 99%, progressively unfreeze later blocks.
5. Re-export to ONNX — deployment path unchanged.

8. Limitations

Concern	Severity	Notes
Only 9 source PDFs (138 pages)	Medium	Small dataset — accuracy estimate has high variance
Synthetic messiness only	Medium	Real degraded scans may behave differently
Pre-training data unknown	Low	Cannot rule out overlap with PaddlePaddle training set
4 rotations per page not independent	Low	Page-level split would give cleaner estimate

The 96.9% figure is directionally reliable but should be re-evaluated on a larger (50+ PDF) held-out test set before making production SLA commitments.

9. Technical Notes

Model format: Paddle 3.x PIR — pass inference.json (not inference.pdmodel) to paddle.inference.Config.

Preprocessing: resize_short(256) → center_crop(224) → normalize(ImageNet) → CHW → batch dim. Simple resize(224,224) produces incorrect results.

ONNX conversion: paddle2onnx opset 17. Constant folding: 282 → 115 nodes.

Optimum fix: input renamed x → pixel_values; output fetch_name_0 → logits; config.json with model_type: resnet required.

Repo: <https://github.com/prabinlamichhane-cell/research-page-orientation>